LOCAL APP

Sri Lanka Super Application Platform

SOFTWARE REQUIREMENTS SPECIFICATION

Version 1.0

Document Revision History

Version	Date	Author	Description
1.0	April 2026	Project Team	Initial Release — Full SRS

Table of Contents

1. Introduction

1.1 Purpose

1.2 Scope

1.3 Definitions & Acronyms

1.4 Intended Audience

1.5 Project Constraints & Timeline

2. Overall Description

2.1 Product Perspective

2.2 User Classes & Roles

2.3 Assumptions & Dependencies

2.4 Technology Stack

3. Functional Requirements

3.1 Authentication & User Management

3.2 Events & Interactions Module

3.3 Landmarks & Attractions Module

3.4 Vendor & Loyalty Module

3.5 Wallet & Point System

3.6 Promo Codes & Discounts

3.7 Lottery System

3.8 Matrimonial Module

3.9 Chat & Group Module

3.10 Admin Panel

4. Non-Functional Requirements

5. System Architecture

6. Database Design

7. API Design Overview

8. Security Requirements

9. UI/UX Requirements

10. Development Sprint Plan (40-Day Timeline)

11. Risk Register

12. Appendices

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the complete functional and non-functional requirements for the LOCAL APP — a Sri Lanka-based super application platform. This document serves as the authoritative reference for the development team, project stakeholders, QA engineers, and DevOps personnel throughout the 40-day development cycle.

This SRS follows IEEE 830 standard conventions adapted for agile delivery, and captures all modules, user roles, data flows, and system behaviors required to build a production-ready, scalable application.

1.2 Scope

LOCAL APP is a full-stack super application targeting the Sri Lankan market, combining cultural event discovery, tourist attraction navigation, vendor loyalty programs, digital wallet infrastructure, a gamified lottery system, and a matrimonial matchmaking platform into a single unified product.

The platform is delivered across two surfaces:

- Web Application (Next.js / Node.js SSR) — for admin, vendors, and desktop users
- Mobile Application — companion app for users on Android/iOS

The system serves three primary roles — Admin, Vendor, and User — and integrates Google Maps, QR-based transactions, KYC workflows, and a peer-to-peer digital wallet.

1.3 Definitions, Abbreviations & Acronyms

Term / Acronym	Definition
SRS	Software Requirements Specification
KYC	Know Your Customer — identity verification process
P2P	Peer-to-Peer — direct user-to-user transfer
QR	Quick Response code used for scan-based transactions
SSR	Server-Side Rendering (Next.js/Node.js)
RBAC	Role-Based Access Control
OTP	One-Time Password for verification

Admin	Platform administrator with full system access
Vendor	Registered business/shop on the platform
User	End consumer / registered member
Dansal	Sri Lankan free food/drink stall during Vesak festival
Shramadana	Sri Lankan community voluntary service event
Toran	Decorative gate/arch erected during festivals
Loyalty Points	Points granted by vendors redeemable in-app
Wallet	In-app digital account for holding balance and points
LKR / Rs	Sri Lankan Rupee — primary currency
Redis	In-memory data store used for caching and sessions
MongoDB	Document-based NoSQL database (primary DB)

1.4 Intended Audience

Audience	Purpose
Development Team	Implementation reference — all functional and technical specs
Project Manager	Scope baseline, timeline adherence, sprint planning
QA Engineers	Test case derivation from functional requirements
UI/UX Designers	Screen flows, component logic, and interaction rules
DevOps / Infra	Architecture, deployment, and environment planning
Business Stakeholders	Feature validation and change management

1.5 Project Constraints & Timeline

Constraint	Detail
Total Duration	40 days (approximately 1 month and 12 days)
Team Structure	Full-stack developers + UI/UX designer
Frontend	Next.js (React-based SSR framework)

Backend	Node.js with Express/Fastify (SSR layer)
Primary Database Cache / Session DB	MongoDB (document store) Redis
Maps Integration	Google Maps Platform (Places, Geocoding, Maps JS API)
Payment Gateway Mobile App	No payment gateway in Phase 1; wallet top-up via manual or future integration Companion mobile app (React Native recommended for code sharing)
Deployment Target	Cloud (AWS / GCP / DigitalOcean) — to be confirmed

2. Overall Description

2.1 Product Perspective

LOCAL APP is a greenfield super application — a single digital destination combining social events, tourism, commerce, and fintech under one platform. The application is analogous to combining features from Facebook Events, Google Maps POI, a digital loyalty card wallet, a fintech wallet, and a matrimonial portal — all tailored for Sri Lankan culture.

The platform operates as a closed-loop financial ecosystem where users can deposit and spend, but cannot withdraw (only vendors can cash out). This design positions the app as a Central Bank-compliant fintech instrument targeting a future credit card license.

2.2 User Classes & Roles

Role	Access Level	Key Capabilities
Admin	Full system access	Approve events, manage landmarks/attractions, review withdrawals, configure lottery, manage all users/vendors, KYC review
Vendor	Business account	Add loyalty programs, promote products, top up wallet, grant loyalty points via QR, request withdrawals, view linked user transactions
User (Consumer)	Standard member	Browse & RSVP events, earn/spend loyalty points, P2P transfers, deposit to wallet, matrimonial profile, lottery participation
Guest	Read-only (public)	View public events and attractions (no wallet, no RSVP without login)

2.3 Assumptions & Dependencies

- Google Maps Platform API key is available and billing enabled before development starts.
- SMS/OTP gateway (e.g., Dialog, Mobitel, or Twilio) is procured for KYC phone verification.
- Email service (SendGrid / SES) is configured for transactional emails.
- A cloud hosting environment is available from Day 1 for CI/CD pipelines.
- The 40-day timeline assumes no major scope additions after SRS sign-off.
- Mobile app uses the same backend APIs — no separate backend development required.
- No payment gateway integration in Phase 1; wallet top-up flow will be placeholder or manual.

2.4 Technology Stack

Layer	Technology	Purpose
Frontend (Web)	Next.js 14+ (App Router)	SSR, SEO-optimized public pages, admin/vendor dashboards
Backend	Node.js + Express/Fastify	REST API, business logic, background jobs
Primary Database	MongoDB (Mongoose ODM)	User data, events, transactions, profiles
Cache / Queue	Redis	Sessions, rate limiting, job queues, real-time data
Real-Time	Socket.io / Redis Pub-Sub	Chat module, live notifications
Maps	Google Maps Platform	Location selection, nearby events, POI display
Storage	AWS S3 / Cloudflare R2	Media uploads (images, KYC docs)
Authentication	JWT + HttpOnly Cookies	Stateless auth with refresh token rotation
Email	SendGrid / AWS SES	OTP, KYC notifications, event reminders
SMS / OTP	Twilio / Dialog Axiata	Phone verification for KYC
Background Jobs	BullMQ (Redis-backed)	Lottery draws, event expiry, notification dispatch
Mobile	React Native (Expo)	iOS & Android companion app (shared API layer)
DevOps	Docker + GitHub Actions	Containerized deployment, CI/CD

3. Functional Requirements

3.1 Authentication & User Management

3.1.1 User Registration

Two registration paths exist based on the user's intended feature usage:

Registration Type	Required Fields	Unlocks
Basic (Email Verified)	Full name, email, password, phone (optional)	Events, attractions, chat, matrimonial browsing
Financial KYC	NIC front & back photo, full legal name, DOB, phone (OTP verified), bank account number + bank name	Wallet deposit, P2P transfer, lottery, merchant payments

- REQ-AUTH-001: System SHALL send an email verification link upon registration. Account is inactive until email is verified.
- REQ-AUTH-002: KYC documents SHALL be stored encrypted in cloud object storage. Admin reviews via a dedicated KYC queue.
- REQ-AUTH-003: System SHALL NOT allow financial operations until KYC status is APPROVED.
- REQ-AUTH-004: Admin can approve, reject, or request re-submission of KYC with a reason message.

3.1.2 Vendor Registration

- REQ-AUTH-005: Vendor registers with business name, business registration number, address, email, phone, and bank details.
- REQ-AUTH-006: Vendor account requires admin approval before activation.
- REQ-AUTH-007: Vendor profile supports multiple business categories (food, retail, services, etc.).

3.1.3 Login & Session Management

- REQ-AUTH-008: JWT access token (15-minute expiry) + refresh token (30-day expiry) in HttpOnly cookie.
- REQ-AUTH-009: System SHALL support social login (Google OAuth) as a secondary option.
- REQ-AUTH-010: Concurrent session management — admin can force logout all sessions for a user.
- REQ-AUTH-011: Password reset via email link with 1-hour expiry.
- REQ-AUTH-012: Rate limiting on login attempts: 5 failed attempts lock account for 15 minutes.

3.1.4 Role-Based Access Control (RBAC)

Permission	Admin	Vendor	User	Guest
View public events	Y	Y	Y	Y
Create event	Y	Y	Y	N
Approve event	Y	N	N	N
Add landmark/attraction	Y	N	N	N
Add loyalty program	N	Y	N	N
Grant loyalty points	N	Y	N	N
Deposit to wallet	N	Y	Y	N
P2P transfer	N	N	Y	N
Withdraw funds	N	Y (manual)	N	N
Participate in lottery	N	N	Y	N
Matrimonial profile	N	N	Y	N
View all transactions	Y	N	N	N
Configure lottery	Y	N	N	N

3.2 Events & Interactions Module

3.2.1 Event Data Model

Field	Type	Required	Notes
title	String	Y	Max 120 characters
description	String	Y	Rich text, max 2000 chars
category	Enum	Y	musical, dansal, shramadana, cultural, sports, meetup, matrimonial, other
event_type	Enum	Y	free | paid_inapp | paid_external
location	GeoJSON Point	Y	Lat/Lng from Google Maps OR linked landmark/attraction ID
location_label	String	Y	Human-readable address

landmark_ref	ObjectId (ref)	N	Optional link to Landmark/Attraction document
start_datetime	DateTime	Y	ISO 8601
end_datetime	DateTime	Y	Must be > start_datetime
status	Enum	Y	pending | approved | rejected | expired | cancelled
created_by	ObjectId (User)	Y	Creator reference
ticket_price	Number	Conditional	Required if paid_inapp
external_link	URL	Conditional	Required if paid_external
images	Array<String>	N	Up to 5 image URLs
max_attendees	Number	N	Optional cap
tags	Array<String>	N	Searchable tags
is_featured	Boolean	N	Admin can feature an event
chat_enabled	Boolean	N	Toggle event chat group

3.2.2 Event Lifecycle

1. User (any role) creates event via structured form.
2. Event enters PENDING state — visible only to creator and admin.
3. Admin reviews event in moderation queue.
4. Admin APPROVES → event becomes visible on web and mobile.
5. Admin REJECTS → creator receives reason via notification.
6. Event auto-expires after end_datetime — free events are soft-deleted; paid events are archived.

- REQ-EVT-001: Location selection MUST use Google Maps embed. User pins location on map OR selects from existing Landmark/Attraction database.
- REQ-EVT-002: Free events SHALL auto-expire 24 hours after end_datetime and be removed from public feeds.
- REQ-EVT-003: Paid (in-app) events SHALL require user to have wallet balance. Purchase records a transaction.

- REQ-EVT-004: Paid (external) events display a CTA button that opens the external URL in the mobile/web browser.
- REQ-EVT-005: All public events MUST display a map pin at their location.
- REQ-EVT-006: System SHALL support geo-proximity search — return events within a configurable radius (default: 10 km) of a user's current location or a selected attraction.
- REQ-EVT-007: Admin can feature up to N events on the homepage carousel (N configurable).

3.2.3 Nearby Events (Geo-Contextual Discovery)

When a user is browsing an Attraction or Landmark, the system SHALL:

- Query all active events where the event location is within the configured radius of the attraction's coordinates.
- Sort results by distance (nearest first) then by start_datetime.
- Display the list beneath the attraction detail page with a map overlay.
- REQ-EVT-008: Radius is configurable per-attraction by admin (default 10 km, range 1–50 km).

3.3 Landmarks & Attractions Module

3.3.1 Definitions & Hierarchy

The platform separates Landmarks and Attractions in a parent-child hierarchy. A Landmark represents a geographic area or region (the broader place name people refer to in common usage), while an Attraction is a specific visitable site within or associated with that landmark area. Both are stored in the same MongoDB collection with a type field and a parent_landmark_id reference on attraction documents.

Real-world example: 'Sigiriya' is the Landmark (the area/ancient city recognised by name). 'Sigiriya Rock Fortress' is the Attraction — the specific, visitable, ticketed UNESCO World Heritage site within that area. Similarly, 'Hikkaduwa' is a Landmark (coastal town area) while 'Hikkaduwa Coral Reef' and 'Hikkaduwa Beach' are individual Attractions within it.

Type	Definition	Example	User Value
Landmark	A broad geographic area, region, or well-known place name used as a navigational or spatial reference	Sigiriya (the area / region / ancient city)	Spatial anchor; used to group nearby attractions and events; searchable reference point

Attraction	A specific, visitable point of interest within or associated with a landmark area	Sigiriya Rock Fortress (the physical rock & UNESCO site)	Reviews, ratings, event hosting, SEO anchor, visitor information

- REQ-ATT-001: Only admins can CREATE landmarks and attractions.
- REQ-ATT-002: Each landmark/attraction stores: name, type (landmark | attraction), description, coordinates (GeoJSON Point), province, district, city, category, images, operating_hours (for attractions), entry_fee (for attractions), website_url, SEO metadata fields (meta_title, meta_description, seo_slug). Landmarks additionally store a bounding_area (GeoJSON Polygon) to define the geographic region.
- REQ-ATT-003: An Attraction must be linked to a parent Landmark (e.g., Sigiriya Rock Fortress links to Sigiriya landmark/area). This parent-child relationship enables grouping attractions under a landmark on the map and on the discovery feed.
- REQ-ATT-004: Users can submit Reviews and Ratings (1–5 stars) for Attractions only (not raw landmarks). Reviews are moderated (auto-publish unless reported).
- REQ-ATT-005: SEO pages are server-side rendered (Next.js SSR) with unique meta tags per attraction/landmark for Google discoverability.
- REQ-ATT-006: Attractions appear on the app's public map with filterable categories. Landmarks are displayed as region labels or area polygons on the map.
- REQ-ATT-007: Admin can link events to an attraction retroactively.

3.4 Vendor & Loyalty Module

3.4.1 Vendor Profile

- REQ-VND-001: Vendor profile fields: business_name, category, address, coordinates, phone, email, website, logo, cover_image, operating_hours, description.
- REQ-VND-002: Vendor dashboard displays: active loyalty programs, points pool balance, transaction history, linked users summary.

3.4.2 Loyalty Program

A vendor defines one or more loyalty programs:

Field	Type	Notes
program_name	String	e.g., 'Spend & Earn Gold'
earn_rule	Object	spend_amount (LKR) → points_awarded (e.g., 1000 Rs = 10 pts)

redeem_rule	Object	points_required for a defined reward (e.g., 1000 pts = T-shirt)
points_pool	Number	Vendor pre-loads points pool (deducted when given to user)
is_active	Boolean	Vendor can pause/activate
expiry_policy	Enum	never | rolling_days (e.g., points expire 180 days after earn)

- REQ-VND-003: When vendor grants points to a user, the vendor's points_pool is DEBITED and the user's loyalty wallet is CREDITED atomically (MongoDB transaction).
- REQ-VND-004: Points granted by vendors are HELD/LOCKED — they CANNOT be withdrawn by the user. They can only be used for redemption at the same or other configured vendors, or for lottery tickets.
- REQ-VND-005: Vendor tops up their points pool by transferring from their vendor wallet balance. Admin confirms.
- REQ-VND-006: Cashier (vendor employee) scans user QR code → enters purchase amount → system calculates points per earn_rule → grants points pending vendor confirmation.
- REQ-VND-007: If vendor uses an external loyalty system (Nexus-style), the platform exposes a webhook/API endpoint for the external system to POST point grants.

3.4.3 QR Code Mechanism

- REQ-VND-008: Each user has a unique, rotating QR code (refreshed every 24 hours or on demand).
- REQ-VND-009: QR encodes: user_id + timestamp + HMAC signature. Vendor app validates signature server-side.
- REQ-VND-010: Same QR mechanism is used for loyalty point grant, promo code redemption, and event check-in.

3.5 Wallet & Point System

3.5.1 Wallet Architecture

Each entity has a wallet document with sub-ledgers:

Wallet Type	Owner	Sub-Ledgers	Can Withdraw	Can Deposit
User Wallet	Registered User	cash_balance, loyalty_points (locked), bonus_points	N (cash-in only)	Y (via payment gateway)

Vendor Wallet	Vendor Account	cash_balance, points_pool	Y (manual, admin-reviewed)	Y (top-up)

- REQ-WAL-001: All wallet operations (credit, debit, transfer) are recorded as immutable Transaction documents.
- REQ-WAL-002: Transactions have fields: id, from_wallet_id, to_wallet_id, amount, currency (LKR or POINTS), type (deposit | spend | transfer | loyalty_earn | lottery_spend | lottery_win | withdrawal_request | fee), status (pending | completed | failed | reversed), description, created_at.
- REQ-WAL-003: Deposit: User deposits LKR — gateway fee is deducted (e.g., Rs 1000 deposited → Rs 975 credited). Fee amount and percentage are admin-configurable.
- REQ-WAL-004: P2P Transfer: User can transfer cash_balance to another user via phone/username lookup. Transfer is atomic. No fee in Phase 1.
- REQ-WAL-005: User CANNOT withdraw cash to a bank account. Only vendor withdrawal is allowed.
- REQ-WAL-006: Vendor withdrawal is a manual process: vendor raises request → admin reviews → admin marks as processed → bank transfer completed offline → system updated.
- REQ-WAL-007: Spending limit: Each user account has a configurable spending_limit (LKR per day/month). Admin sets default; limit increases over time based on account history. User cannot override this.
- REQ-WAL-008: KYC is MANDATORY before any financial operation. System returns 403 with KYC_REQUIRED error code for unapproved users.

3.5.2 Corporate / Family Accounts

- REQ-WAL-009: A vendor account can be designated as a CORPORATE account.
- REQ-WAL-010: Corporate vendor can top up linked user sub-accounts. The top-up creates a work_balance sub-ledger in the user's wallet (separate from personal balance).
- REQ-WAL-011: Corporate vendor can view transaction summaries (amount, merchant, timestamp) of work_balance transactions — NOT personal balance transactions.
- REQ-WAL-012: A user can bind their existing personal account to a corporate vendor. Binding requires user consent (OTP confirmation). Work transactions are tagged company_id for filtering.

3.6 Promo Codes & Discounts

- REQ-PRO-001: Admin or Vendor can create promo codes with: code (unique string), discount_type (percentage | fixed_LKR), discount_value, min_purchase, max_uses, valid_from, valid_until, applicable_to (events | vendors | all).
- REQ-PRO-002: Users enter promo code at event checkout or scan QR-encoded promo code at vendor POS.
- REQ-PRO-003: System validates code, checks usage limits and expiry, applies discount to the transaction.
- REQ-PRO-004: Used codes are tracked per user (prevent double use). Admin dashboard shows redemption analytics.

3.7 Lottery System

3.7.1 Lottery Configuration

Parameter	Type	Default	Notes
lottery_name	String	—	Display name
start_date	DateTime	—	When ticket purchase opens
end_date	DateTime	—	Auto-draws at this datetime
ticket_cost_points	Number	—	Points per ticket (e.g., 1 pt = 1 ticket)
platform_fee_pct	Number	20%	% kept by platform
winner_count	Number	5	Configurable 1–10
prize_distribution	Array<Object>	40/20/13.33/13.33/13.33	% per winner position
prize_type	Enum	points	points only (Phase 1)
draw_method	Enum	random	Cryptographically random draw

- REQ-LOT-001: User purchases tickets by spending loyalty_points. System checks sufficient balance before deduction.
- REQ-LOT-002: Tickets are stored with user_id, lottery_id, ticket_numbers (array), purchased_at.
- REQ-LOT-003: At end_date, a scheduled BullMQ job triggers the draw.
- REQ-LOT-004: Draw uses cryptographically secure randomness (Node.js crypto.randomInt). Each ticket number has equal probability.
- REQ-LOT-005: Total points pool = sum of all tickets sold in that lottery round.

- REQ-LOT-006: Platform fee is deducted first. Remaining pool is distributed per prize_distribution percentages.
- REQ-LOT-007: Winners receive points directly to their wallet. System creates transactions of type lottery_win.
- REQ-LOT-008: Results are published on the app. All participants receive a push notification.
- REQ-LOT-009: Admin can run multiple simultaneous lotteries (e.g., daily micro-lottery + monthly mega-lottery).

3.8 Matrimonial Module

3.8.1 Profile

- REQ-MAT-001: Matrimonial profile is SEPARATE from the main user profile. User opts-in.
- REQ-MAT-002: Profile fields: gender, date_of_birth, religion, ethnicity, caste (optional), height, education, occupation, income_range, district, city, about_me (500 chars), hobbies, family_details, expectations, photos (up to 6, first is primary).
- REQ-MAT-003: Photos require manual moderation (admin queue) before appearing publicly.
- REQ-MAT-004: Profile has privacy settings: show_full_name (Y/N), show_photos_to (all | matches_only | none).

3.8.2 Search & Browse

- REQ-MAT-005: Search filters: gender, age_range, religion, district, ethnicity, height_range, education.
- REQ-MAT-006: First 20 results per search are shown for free (paginated 20 per page).
- REQ-MAT-007: To view more profiles or send an expression of interest, user must spend in-app coins (configurable by admin, e.g., 50 coins per profile view unlock).
- REQ-MAT-008: System tracks profile view counts. Viewed profiles are stored in user history (cannot re-charge for viewing same profile within 7 days).

3.8.3 Matrimonial Events

- REQ-MAT-009: Speed dating / meetup events are a SUBSET of the main Events module with category = matrimonial.
- REQ-MAT-010: Event organizer is an ANNOUNCER role (sub-role of Vendor or Admin).
- REQ-MAT-011: Registration for matrimonial events requires in-app coin payment.
- REQ-MAT-012: No virtual matching — the platform facilitates physical attendance. Post-event, attendees can connect via the app chat.

3.9 Chat & Group Module

- REQ-CHT-001: Each event has an optional auto-created chat group (enabled if chat_enabled = true on event).
- REQ-CHT-002: Event creator can invite users or make group join-link public.
- REQ-CHT-003: Group types: event_group (tied to event lifecycle), direct_message (1:1 between users).
- REQ-CHT-004: Messages are stored in MongoDB (chat_messages collection) with: group_id, sender_id, content, type (text | image | system), created_at, read_by (array).
- REQ-CHT-005: Real-time delivery via Socket.io rooms. Message history paginated (50 per load).
- REQ-CHT-006: Event groups auto-archive 7 days after event end_datetime. Archived groups are read-only.
- REQ-CHT-007: Users can report messages. Reported messages are reviewed by admin.
- REQ-CHT-008: Push notifications for new messages when user is not active in the app.

3.10 Admin Panel

3.10.1 Dashboard KPIs

KPI	Description
Total Users	Registered users (verified / unverified / KYC approved)
Active Events	Currently live approved events
Pending Approvals	Events + KYC + vendor registrations awaiting review
Wallet Activity	Total deposits, spend, P2P transfers in last 7/30 days
Lottery Status	Active lotteries, tickets sold, pool size
Vendor Count	Active / pending / suspended vendors

3.10.2 Admin Functions

- Event moderation queue with approve / reject / edit-request actions.
- KYC review queue — view uploaded documents, approve/reject with reason.
- Landmark & Attraction CRUD with Google Maps coordinate picker.
- Lottery configuration and management (create, monitor, force-draw).
- Vendor withdrawal processing queue.
- Promo code management (create, deactivate, view analytics).

- User spending limit configuration (global default + per-user override).
- Platform fee configuration (deposit fee %, withdrawal fee %).
- Matrimonial profile moderation (photo approval).
- Reported content review (chat messages, reviews).
- System-wide push notification broadcast.
- Analytics: event views, app usage, wallet volumes, lottery participation, matrimonial activity.

4. Non-Functional Requirements

ID	Category	Requirement
NFR-001	Performance	API response time < 200ms for 95th percentile under normal load (< 500 concurrent users)
NFR-002	Performance	Map and event feed pages SHALL achieve Time to Interactive (TTI) < 3 seconds on 4G mobile
NFR-003	Scalability	System architecture SHALL support horizontal scaling — stateless API servers behind a load balancer
NFR-004	Availability	Target 99.5% uptime (approx. 44 hours downtime/year). Maintenance windows communicated 24h in advance
NFR-005	Security	All data in transit encrypted via TLS 1.2+. KYC documents encrypted at rest (AES-256)
NFR-006	Security	OWASP Top 10 mitigations applied. Dependency vulnerability scanning in CI pipeline
NFR-007	Data Integrity	All financial transactions use MongoDB multi-document ACID transactions. No partial writes allowed
NFR-008	Auditability	All financial transactions are immutable. No delete operations on transaction records — only status updates
NFR-009	Compliance	KYC flow designed to meet Central Bank of Sri Lanka directives for e-money operators
NFR-010	Usability	Application SHALL support Sinhala, Tamil, and English language UI (i18n-ready architecture)
NFR-011	SEO	All attraction and event public pages SHALL be server-side rendered with unique meta tags for Google indexing
NFR-012	Caching	Redis caches: event feeds (TTL 60s), attraction list (TTL 300s), user sessions (TTL 24h)
NFR-013	Mobile	Mobile app SHALL support Android 9+ and iOS 14+
NFR-014	Monitoring	Application logs sent to centralized logger (Logtail / Datadog). Error alerting via Slack/email

NFR-015	Backup	MongoDB daily automated backups. Point-in-time recovery available for last 7 days

5. System Architecture

5.1 High-Level Architecture

The application follows a layered, service-oriented architecture deployed in Docker containers:

Layer	Components	Technology
Client (Web)	Next.js App Router pages, SSR for public content, client-side for authenticated dashboards	Next.js 14, TailwindCSS, React Query
Client (Mobile)	React Native app consuming the same REST API	Expo, React Native, TailwindRN
API Gateway	Rate limiting, auth validation, request routing	Nginx / Express middleware
Application Layer	REST API handlers, business logic, RBAC enforcement	Node.js, Express/Fastify
Background Jobs	Event expiry, lottery draw, notification dispatch, KYC reminders	BullMQ workers (Redis queues)
Cache Layer	Session tokens, feed caches, rate limit counters, Socket.io adapter	Redis 7+
Database Layer	Primary data store for all entities	MongoDB 7+ (replica set)
File Storage	User uploads, KYC documents, event/attraction images	AWS S3 / Cloudflare R2
Real-Time Layer	Chat, live notifications	Socket.io + Redis Pub/Sub
External Services	Google Maps, SMS gateway, email service	3rd party APIs

5.2 Deployment Architecture

- All services containerized with Docker and orchestrated via Docker Compose (dev) or Kubernetes/ECS (prod).
- Next.js app deployed as Node.js SSR server (not static export) to support server-side data fetching.
- MongoDB deployed as a 3-node replica set to enable ACID multi-document transactions and high availability.
- Redis deployed as single instance (Phase 1); Sentinel/Cluster for Phase 2 scaling.
- Nginx reverse proxy handles SSL termination, static asset caching, and upstream routing.

- GitHub Actions CI/CD pipeline: lint → test → build Docker image → push to registry → deploy.

6. Database Design

6.1 MongoDB Collections

Collection	Primary Fields	Key Indexes
users	_id, email, phone, password_hash, role, kyc_status, created_at	email (unique), phone (unique)
vendor_profiles	_id, user_id, business_name, category, location (2dsphere), status	user_id (unique), location (2dsphere)
wallets	_id, owner_id, owner_type, cash_balance, loyalty_points, work_balance, points_pool	owner_id (unique)
transactions	_id, from_wallet, to_wallet, amount, currency, type, status, created_at	from_wallet, to_wallet, created_at, type
events	_id, title, category, location (2dsphere), status, start_datetime, end_datetime, created_by	location (2dsphere), status, start_datetime
landmarks	_id, name, type (landmark|attraction), parent_landmark_id (for attractions), location (2dsphere), bounding_area (GeoJSON Polygon, for landmarks), district, category, seo_slug	location (2dsphere), seo_slug (unique), parent_landmark_id
loyalty_programs	_id, vendor_id, name, earn_rule, redeem_rule, points_pool, is_active	vendor_id
loyalty_grants	_id, program_id, vendor_id, user_id, points, created_at	user_id, vendor_id
lotteries	_id, name, start_date, end_date, ticket_cost, status, pool_total, winners	status, end_date
lottery_tickets	_id, lottery_id, user_id, ticket_count, points_spent, purchased_at	lottery_id, user_id
matrimonial_profiles	_id, user_id, district, religion, dob, status, photos	user_id (unique), district
promo_codes	_id, code, discount_type, discount_value, uses_count, max_uses, valid_until	code (unique)

chat_groups	_id, event_id, members, type, created_at	event_id
chat_messages	_id, group_id, sender_id, content, type, created_at	group_id, created_at
kyc_submissions	_id, user_id, nic_front_url, nic_back_url, status, reviewed_by, created_at	user_id, status
reviews	_id, landmark_id, user_id, rating, content, status, created_at	landmark_id, user_id

6.2 Redis Key Patterns

Key Pattern	Value	TTL	Purpose
session:{userId}	JWT payload JSON	24h	Session store
ratelimit:login:{ip}	Integer (attempt count)	15min	Brute-force protection
feed:events:{hash}	JSON event array	60s	Event feed cache
feed:attractions	JSON attractions array	300s	Attractions list cache
qr:{userId}	HMAC token	24h	User QR code validation
lottery:tickets:{lotteryId}	Sorted set (userId:score)	TTL = lottery end	Ticket pool for draw
chat:online:{groupId}	Set of userIds	Session-based	Online presence in chat
notif:queue:{userId}	List of notification objects	7 days	Undelivered notifications

7. API Design Overview

7.1 Base URL & Versioning

All API routes are prefixed: /api/v1/

Authentication: Bearer token in Authorization header OR HttpOnly cookie (web).

7.2 Core Endpoint Groups

Group	Base Path	Key Endpoints
Authentication	/api/v1/auth	POST /register, POST /login, POST /refresh, POST /logout, POST /kyc/submit, GET /kyc/status
Users	/api/v1/users	GET /me, PATCH /me, GET /:id/profile, GET /search
Events	/api/v1/events	GET / (list, filter, geo), POST / (create), GET /:id, PATCH /:id, DELETE /:id, POST /:id/attend, POST /:id/approve (admin)
Landmarks	/api/v1/landmarks	GET / (list, map), POST / (admin only), GET /:id, PATCH /:id (admin), GET /:id/events (nearby), POST /:id/review
Vendors	/api/v1/vendors	GET /me, POST /loyalty-programs, GET /loyalty-programs, POST /grant-points, GET /users (linked)
Wallet	/api/v1/wallet	GET /balance, POST /deposit, POST /transfer, GET /transactions, POST /withdraw-request (vendor)
Lottery	/api/v1/lottery	GET /active, POST /:id/buy-tickets, GET /:id/results, POST / (admin create)
Promo	/api/v1/promo	POST /validate, POST /redeem, POST / (admin/vendor create)
Matrimonial	/api/v1/matrimonial	GET /profiles, POST /profile (create), PATCH /profile, POST /unlock/:profileId
Chat	/api/v1/chat	GET /groups, GET /groups/:id/messages, POST /groups/:id/messages
Admin	/api/v1/admin	GET /dashboard, GET /events/pending, PATCH /events/:id/status, GET /kyc/queue, PATCH /kyc/:id, GET /withdrawals, PATCH /withdrawals/:id
Notifications	/api/v1/notifications	GET / (user's notifications), PATCH /:id/read

7.3 Standard Response Envelope

All API responses follow a consistent envelope:

Field	Type	Description
success	Boolean	true for 2xx, false for errors
data	Object | Array | null	Response payload
message	String	Human-readable status message
error_code	String | null	Machine-readable error code (e.g., KYC_REQUIRED, INSUFFICIENT_BALANCE)
pagination	Object | null	{ page, limit, total, total_pages } for list endpoints
timestamp	String	ISO 8601 server timestamp

8. Security Requirements

ID	Requirement
SEC-001	HTTPS enforced on all routes. HTTP requests redirected to HTTPS with 301.
SEC-002	JWT tokens signed with RS256 (asymmetric). Private key stored in environment variables, never committed to source.
SEC-003	Passwords hashed with bcrypt (cost factor 12). No plaintext passwords stored anywhere.
SEC-004	KYC document URLs are pre-signed S3 URLs with 15-minute expiry. Not publicly accessible.
SEC-005	QR code tokens include HMAC-SHA256 signature. Vendor app validates signature server-side before crediting points.
SEC-006	All financial operations validated server-side. Client-side amounts are NEVER trusted for wallet operations.
SEC-007	CORS policy: web app domain whitelisted. Mobile app uses certificate pinning.
SEC-008	Rate limiting on all public endpoints: 100 req/min per IP for anonymous, 300 req/min for authenticated.
SEC-009	Input validation and sanitization on all user-supplied data (express-validator / Joi). SQL/NoSQL injection prevention.
SEC-010	XSS prevention: Content Security Policy headers set. User-generated HTML content is sanitized with DOMPurify equivalent.
SEC-011	Admin panel restricted to whitelisted IP range (configurable) AND 2FA (TOTP).
SEC-012	Sensitive operations (withdrawal, KYC approval, lottery draw) are logged with admin user ID for full audit trail.
SEC-013	Dependency vulnerability scanning via npm audit / Snyk in CI pipeline. Critical vulnerabilities block deployment.

9. UI/UX Requirements

ID	Requirement
UX-001	Mobile-first responsive design. All web pages must render correctly on 320px–1920px viewports.
UX-002	Navigation: bottom tab bar on mobile (Home, Events, Map, Wallet, Profile). Top navigation on desktop.
UX-003	Event cards display: title, category badge, date/time, location (city), event type (Free/Paid), thumbnail.
UX-004	Map view: Google Maps with clustered event and attraction pins. Tap pin → bottom sheet with brief details + CTA.
UX-005	Wallet screen shows: cash_balance, loyalty_points, recent transactions (last 5), action buttons (Deposit, Transfer, Scan QR).
UX-006	QR code screen is prominently accessible from wallet (tap to show full-screen QR with user's display name).
UX-007	Matrimonial profile cards show: first name, district, age range, occupation, one photo. Full details revealed after unlock.
UX-008	Admin panel: data table with pagination, search, bulk actions. Responsive sidebar navigation.
UX-009	Onboarding flow: 3-screen tour for new users highlighting events, wallet, and loyalty features.
UX-010	Loading states: skeleton screens for content areas. No blank white screens during data fetch.
UX-011	Error states: user-friendly messages for network errors, empty states (no events, no transactions).
UX-012	Push notification permission request shown after first successful login (not at app launch).
UX-013	Dark mode support (follows device system preference).
UX-014	Accessibility: minimum contrast ratio 4.5:1, touch targets minimum 44x44px, screen reader labels on interactive elements.

10. Development Sprint Plan (40-Day Timeline)

10.1 Sprint Overview

Sprint	Days	Focus Area	Key Deliverables
Sprint 0	Days 1–3	Setup & Architecture	Repo, CI/CD, Docker, MongoDB + Redis setup, env config, base Next.js + API project scaffold, shared component library init
Sprint 1	Days 4–9	Auth + Core Models	User/Vendor registration, email OTP, KYC submission flow, JWT auth, RBAC middleware, MongoDB schema definitions for all collections
Sprint 2	Days 10–16	Events Module + Maps	Event CRUD, admin approval flow, Google Maps integration, location selection, geo-proximity search, event expiry job, public SSR event pages
Sprint 3	Days 17–22	Landmarks + Wallet Core	Landmark/Attraction CRUD (admin), attraction public pages (SEO), reviews, wallet document creation, deposit flow, P2P transfer, transaction history
Sprint 4	Days 23–27	Vendor + Loyalty + QR	Vendor dashboard, loyalty program CRUD, QR generation + validation, cashier points grant flow, points pool management, promo code engine
Sprint 5	Days 28–32	Lottery + Matrimonial	Lottery configuration, ticket purchase, BullMQ draw job, prize distribution, matrimonial profile, search, coin unlock, photo moderation queue
Sprint 6	Days 33–36	Chat + Admin Panel	Socket.io chat (event groups + DMs), admin dashboard KPIs, moderation queues (events, KYC, withdrawals, content), analytics views
Sprint 7	Days 37–40	QA, Polish, Launch Prep	End-to-end testing, performance audit, security scan, bug fixes, staging deployment, UAT with stakeholders, production deployment checklist

10.2 Critical Path Items

- Google Maps API key procurement — must be done BEFORE Sprint 2 (Day 10).
- MongoDB Atlas cluster creation — must be done BEFORE Sprint 1 (Day 4).

- KYC document storage (S3/R2) — must be configured BEFORE Sprint 1 end (Day 9).
- BullMQ + Redis worker infrastructure — must be operational BEFORE Sprint 5 (Day 28) for lottery draw.
- UI/UX wireframes for all major flows — must be completed during Sprint 0–1 (Days 1–9) to avoid blocking frontend.

10.3 Team Allocation (Recommended)

Role	Headcount	Primary Responsibility
Full-Stack Lead	1	Architecture, API design, code review, Sprint 0 setup
Backend Developer	1–2	Node.js API, background jobs, wallet engine, lottery, socket
Frontend Developer	1–2	Next.js web app, admin panel, component library
Mobile Developer	1	React Native mobile companion app
UI/UX Designer	1	Wireframes (Figma), design system, component specs
QA Engineer	1	Test cases, Sprint 7 end-to-end testing, regression
DevOps (part-time)	0.5	Docker, CI/CD, cloud infra, monitoring setup

11. Risk Register

ID	Risk	Probability	Impact	Mitigation
RSK-001	Google Maps billing exceeds budget	Medium	High	Set API usage quotas; cache geocoding results in MongoDB; use static maps where dynamic not needed
RSK-002	40-day timeline insufficient for full scope	High	High	Prioritize core wallet + events for MVP; defer matrimonial and lottery to Phase 2 if needed; daily standups to track velocity
RSK-003	MongoDB transactions cause performance bottleneck	Medium	Medium	Profile transactions early (Sprint 3); use optimistic concurrency where possible; Redis for atomic counters
RSK-004	KYC document regulatory compliance	Medium	High	Consult Central Bank directives before KYC schema finalization; build flexible admin-configurable KYC fields
RSK-005	Payment gateway integration delayed	High	Medium	Phase 1 scoped without payment gateway; wallet top-up uses manual/voucher flow. Gateway added in Phase 2
RSK-006	QR token replay attacks	Low	High	Rotating QR tokens (24h TTL) + HMAC + one-time-use nonces stored in Redis for point grants
RSK-007	Scope creep from stakeholders	High	High	SRS sign-off required before dev starts; change request process defined; all post-SRS changes evaluated for timeline impact
RSK-008	Real-time chat scalability on single Redis instance	Low	Medium	Phase 1 single Redis node sufficient for MVP load. Redis Cluster added in Phase 2 scaling plan

RSK-009	Data privacy for matrimonial module	Medium	High	Profile photos moderated; full profile data only shown to paying users; no raw phone/email exposed via API
RSK-010	Lottery system fairness disputes	Low	Medium	All draw results logged with cryptographic seed; audit log available to admin; results published publicly with ticket numbers

12. Appendices

Appendix A: Event Category Reference

Category Code	Display Name	Description
musical	Musical Show	Live music concerts, cultural performances, DJ events
dansal	Dansal	Free food/drink stalls during Vesak and other Buddhist festivals
shramadana	Shramadana	Community voluntary service events (cleaning, planting, building)
cultural	Cultural Event	Traditional dance, drama, art exhibitions, heritage events
sports	Sports Event	Cricket, football, athletics, water sports, esports
meetup	Community Meetup	Professional meetups, tech events, social gatherings
matrimonial	Matrimonial Event	Speed dating, match-making meetups organized by the platform
religious	Religious Event	Poya day events, church services, kovil celebrations, mosque events
other	Other	Any event not fitting the above categories

Appendix B: Transaction Type Reference

Type Code	Description	Debit	Credit
deposit	User tops up wallet via payment gateway	Gateway account	User wallet
deposit_fee	Platform fee on deposit	User wallet	Platform wallet
p2p_transfer	User-to-user money transfer	Sender wallet	Receiver wallet
event_ticket	User buys paid event ticket	User wallet	Vendor/Platform wallet
loyalty_earn	Vendor grants loyalty points to user	Vendor points pool	User loyalty points

loyalty_redeem	User redeems points at vendor	User loyalty points	Vendor (discount)
lottery_buy	User purchases lottery tickets with points	User loyalty points	Lottery pool
lottery_win	User wins lottery prize	Lottery pool	User wallet (points)
withdrawal_request	Vendor requests withdrawal to bank	Vendor wallet	Pending (held)
withdrawal_complete	Admin completes vendor withdrawal	Held amount	Bank (offline)
promo_discount	Promo code applied at checkout	Promo budget	User (discount)
corp_topup	Corporate vendor tops up employee wallet	Corporate vendor wallet	Employee work_balance
matrimonial_unlock	User unlocks matrimonial profile	User wallet (coins)	Platform wallet

Appendix C: Error Code Reference

Error Code	HTTP Status	Description
KYC_REQUIRED	403	Financial operation attempted before KYC approval
INSUFFICIENT_BALANCE	400	Wallet balance too low for requested operation
SPENDING_LIMIT_EXCEEDED	400	Transaction would breach user spending limit
EVENT_NOT_APPROVED	403	Event not yet approved by admin
INVALID_QR_TOKEN	401	QR token expired, tampered, or already used
PROMO_EXPIRED	400	Promo code has passed its valid_until date
PROMO_MAX_USES	400	Promo code usage limit reached
LOTTERY_CLOSED	400	Lottery ticket purchase window has closed
PROFILE_LOCKED	402	Matrimonial profile requires coin unlock
VENDOR_INACTIVE	403	Vendor account is suspended or pending approval
DUPLICATE_EMAIL	409	Email already registered in the system
TOKEN_EXPIRED	401	JWT access token expired — refresh required

RATE_LIMITED	429	Too many requests — try again after backoff period

Appendix D: KYC Field Requirements

Field	Format	Validation Rule
nic_number	String	Sri Lanka NIC: 9 digits + V/X, or 12 digits (new format). Regex validated.
nic_front_image	JPG/PNG URL	File < 5MB, min resolution 400x300px, JPEG/PNG only
nic_back_image	JPG/PNG URL	Same as front image constraints
full_legal_name	String	Min 2 words, max 100 chars, letters and spaces only
date_of_birth	Date (YYYY-MM-DD)	Must be 18+ years from current date
phone_number	String	Sri Lanka format: +94 7X XXXXXXX. OTP verified.
bank_account_number	String	10–16 digits. Bank name required alongside.
bank_name	Enum	From predefined list of Sri Lankan licensed banks
address	String	Min 10 chars, max 255 chars

END OF DOCUMENT

LOCAL APP SRS v1.0 | Confidential | April 2026